

Chance-Constrained Task Offloading for Reliability Guarantees in Multi-Tenant Networks

Wei Huang, Richard Combes, Andrea Araldo, Hind Castel-Taleb, Badii Jouaber

▶ **To cite this version:**

Wei Huang, Richard Combes, Andrea Araldo, Hind Castel-Taleb, Badii Jouaber. Chance-Constrained Task Offloading for Reliability Guarantees in Multi-Tenant Networks. IEEE International Conference on Communications(ICC), May 2026, Glasgow, United Kingdom. pp.1-6, ⟨10.1109/ICC59461.2026.11586921⟩. ⟨hal-05705287⟩

HAL Id: hal-05705287

https://hal.science/hal-05705287v1

Submitted on 27 Jul 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

Chance-Constrained Task Offloading for Reliability Guarantees in Multi-Tenant Networks

Wei Huang
L2S, CentraleSupelec, Paris-Saclay University
91190 Gif-sur-Yvette, France
wei.huang@centralesupelec.fr

Richard Combes
L2S, CentraleSupelec, Paris-Saclay University
91190 Gif-sur-Yvette, France
richard.combes@centralesupelec.fr

Andrea Araldo
SAMOVAR, Télécom SudParis, IP-Paris
91120 Palaiseau, France
andrea.araldo@telecom-sudparis.eu

Hind Castel-Taleb
SAMOVAR, Télécom SudParis, IP-Paris
91120 Palaiseau, France
hind.castel@telecom-sudparis.eu

Badii Jouaber
SAMOVAR, Télécom SudParis, IP-Paris
91120 Palaiseau, France
badii.jouaber@telecom-sudparis.eu

***Abstract*—We study the problem of task offloading under stochastic delays with reliability guarantees. We introduce the Chance-Constrained Multi-Armed Bandit (CCMAB) framework, where each tenant selects a server to minimize cost while ensuring its experienced delay satisfies a probabilistic latency constraint. We propose the Optimal Chance-Constrained (O2C) algorithm, which learns delay distributions and adapts allocation to meet chance constraints with high probability. We establish its admissibility and derive logarithmic regret bounds, proving strong performance guarantees. Experiments on multi-tenant offloading scenarios show that O2C achieves both reliability guarantees and low cost.**

***Index Terms*—Chance constraints, Task offloading, Multi-armed bandits**

I. Introduction

Task offloading in cloud-edge systems involves dynamically allocating tenants to servers to minimize processing costs and latency. However, stochastic delays and resource contention make deterministic guarantees non-trivial. Reliability requirements (e.g., Service Level Agreements) often demand that the delay stay below a threshold with high probability. This motivates a chance-constrained approach. Chance constraints are a fundamental concept in decision-making under uncertainty, which are also known as probabilistic constraints. In this work, we introduce the chance-constrained multi-player multi-armed bandit in the stochastic setting for task offloading, where not only the cost is associated with a server, but also a stochastic delay of unknown distribution. In addition, there is a chance constraint on the delay that needs to be satisfied for each tenant. Our work aims to find an optimal decision for tenants to offload their tasks to minimize costs while satisfying all reliability constraints.

We propose a novel strategy based on optimism in the face of uncertainty which is known for its efficiency in balancing exploration and exploitation in stochastic environments. Our novel approach allows to accommodate chance constraints with multiple tenants and we apply it to the task offloading problem. The aim is to optimize task offloading decisions while ensuring that the experienced delay remains within acceptable limits with high probability. Note that tenants' selections factor in resource congestion and delay and will be affected when multiple tenants choose the same server. Our contributions are threefold:

- We formulate the chance-constrained multi-armed bandit problem with multiple tenants, incorporating stochastic delays and known costs. We develop an optimistic algorithm, which we call the optimal chance constrained (O2C) algorithm, to find the optimal solution while satisfying all chance constraints. A sort of implicit coordination is realized as the central controller learns to distribute tenants' loads on different servers while avoiding interferences among them.
- We derive a theoretical upper bound on the regret of the O2C algorithm, demonstrating its convergence to the optimal decision.
- We validate our theoretical results through numerical experiments. Our experiments are fully reproducible, and the code is publicly available [1].

The rest of the paper is organized as follows. In Section II, we review related work including multi-armed bandits, structured bandits, chance-constrained optimization, and task offloading. Section III presents the problem formulation. We show our proposed O2C algorithm in Section IV and provide a regret upper bound for O2C in Section V. Section VI details our numerical experiments and results. Finally, Section VII concludes the paper with a discussion of potential future directions.

II. Related work

The classical MAB problem, introduced by [1], involves a single player selecting from a set of arms, each with an unknown reward distribution. The goal is to maximize the cumulative reward over time by choosing arms based on past observations. Notable algorithms addressing this problem include the Upper Confidence Bound (UCB) family [2], [3], which provides a robust framework for balancing exploration and

[1] https://github.com/Free-Wei/offloading

exploitation. Extensions to the classical MAB problem include the contextual bandit, where additional information (context) is available at each decision point [4], and the adversarial bandit, where rewards are determined by an adversary rather than a fixed distribution [5]. However, these models typically assume a single player and do not consider constraints. In practice, the player needs to maximize cumulative reward while satisfying certain constraints. [6], [7] work on knapsack bandits, where additional random cost is consumed when pulling arms. The goal is to maximize cumulative reward before the total budget is fully consumed. [8] studied the setting closest to our work, where they propose constrained confidence bounds on average delay (arms are associated with random delays and arrival rates) under constrained stochastic bandits. However, we instead focus on the Cumulative Distribution Function (CDF) of the delay of dependent arms (arms are associated with random delays and constant costs). Our goal is to minimize cumulative cost while satisfying chance constraints which impose that the CDF of delays of pulled arms is higher than a certain threshold.

Chance-constrained optimization is a critical methodology in decision-making under uncertainty, introduced by [9] to ensure that certain constraints are satisfied with high probability. This concept has been extensively applied across various domains, such as day-ahead unit commitment in power systems [10], decentralized renewable energy systems [11] and the vehicle routing problem [12]. These studies emphasize the importance of maintaining reliability and robustness in stochastic environments.

Task offloading, particularly in cloud and edge computing, involves deciding where to execute tasks to optimize performance metrics such as latency, cost, and reliability. Prior work has examined various heuristics and optimization techniques for task offloading, often focusing on deterministic settings [13], [14]. More recent research has explored stochastic models to account for uncertain task execution times and network conditions [15]. Additionally, instead of considering chance constraints, some works [16]–[18] jointly optimize task offloading and resource allocation with constraints such that the sum of energy or computation consumption under their strategy remains below its capacity.

Our work builds on these foundations by addressing a multi-player, chance-constrained MAB problem with stochastic delays in the context of task offloading. Unlike previous studies, we explicitly model the impact of resource contention on delays and ensure that the experienced delay remains within probabilistic bounds. Our approach utilizes the principle of optimism under uncertainty (used by algorithms such as KL-UCB) to deliver a robust solution that strikes a balance between exploration and exploitation while satisfying chance constraints.

III. Model

We now introduce our novel stochastic bandit setting, which we call Chance-Constrained Multi-Armed Bandit (CCMAB). Consider K servers and M tenants. At each time $t \in [T]$, where T is the time horizon, a central controller selects a server denoted by $x_m(t) \in \{1, ..., K\}$ for each tenant $m \in \{1, ..., M\}$. When tenant m is associated with server k, a constant cost $C_{m,k}$ and a random delay $d_{m,k}(t)$ are experienced. We assume that such delay is proportional to number $G_k(t) = \sum_{m=1}^{M} \mathbb{1}(x_m(t) = k)$ of tenants associated to the same server. This assumption fits well the cases where a certain resource is shared uniformly among multiple tenants, which is the case, for instance, under the processor sharing discipline [19]. In other words, denoting by random variable $\mathcal{D}_{m,k}$ the delay experienced by tenant m if this tenant were the only one associated with server k, we can write:

$$d_{m,k}(t) = G_k(t) \cdot D_{m,k}(t), \tag{1}$$

where $D_{m,k}(t)$ is the realization of random variable $\mathcal{D}_{m,k}$, whose distribution is unknown.

At time t, once tenant $m \in [M]$ is associated with server $k \in [K]$, a deterministic cost $C_{m,k}$ is suffered by the system and a delay realization $d_{m,k}(t)$ is experienced. The decision is made depending on the past observations. To guarantee quality of service (QoS), decisions must ensure that the delay $d_{m,k}(t)$ stays below a threshold L^m with a probability of at least R^m. This is useful in systems where uncertainties exist (e.g., AWS Service Level Agreements [20], Predictive Resource Allocation in wireless communication [21])

$$\mathbb{P}(d_{m,k}(t) \leq L^m) \geq R^m. \tag{2}$$

L^m and R^m are referred to as latency and reliability requirements, respectively, and are known to the central controller. Via (1), this constraint can be written as $F_{\mathcal{D}_{m,k}}(\frac{L^m}{G_k(t)}) \geq R^m$ where $F_{\mathcal{D}_{m,k}}$ denotes the CDF of random variable $\mathcal{D}_{m,k}$. Note that at time t, if tenant m is associated with server k and violates their constraint (2), the corresponding cost $C_{m,k}$ is still incurred by the system. The objective of the central controller is to minimize the expected total cost while ensuring that all chance constraints are satisfied. Let $\mathcal{X}$ denote the feasible allocation set as follows:

$$\mathcal{X} = \left\{ \mathbf{x} = \{x_1, \ldots, x_M\} \mid F_{\mathcal{D}_{m,x_m}} \left(\frac{L^m}{G_{x_m}} \right) \geq R^m, \forall m \right\}$$

We aim to find a decision strategy that minimizes overall cost while satisfying chance constraints, within time horizon T

$$\begin{aligned} \min \quad & \sum_{t=1}^{T} \sum_{m=1}^{M} C_{m,x_m(t)} \\ \text{s.t.} \quad & \frac{1}{T} \sum_{t=1}^{T} F_{\mathcal{D}_{m,x_m(t)}} \left(\frac{L^m}{G_{x_m(t)}} \right) \geq R^m, \forall m \end{aligned} \tag{3}$$

IV. Algorithms

In this section, we propose the O2C (Optimal Chance Constrained for Stochastic Bandits) algorithm, which is based on the principle of optimism in the face of uncertainty. We use notation $f(t) = \log t + 3 \log \log t$, and

$$KL(p, q) = p \log \left(\frac{p}{q} \right) + (1 - p) \log \left(\frac{1 - p}{1 - q} \right)$$

the Kullback-Leibler divergence between Bernoulli distributions of parameters p and q, respectively. Solving optimization problem (3) is non-trivial since we need not only to estimate delay distribution $\mathcal{D}_{m,k}$ for each tenant m and server k, but also to understand how many tenants $G_{m,k}$ they can tolerate when choosing the same server k while achieving minimum cost. In order to satisfy all chance constraints, in the O2C algorithm, we first need to estimate the cumulative distribution function $F_{\mathcal{D}_{m,k}}$ of the delay experienced by tenant m when it is associated with server k, by some estimate $\tilde{F}_{\mathcal{D}_{m,k}}$. This estimation is necessary because the distribution of $\mathcal{D}_{m,k}$ is initially unknown. To this aim, we denote a binary variable $x_{m,k} = \mathbb{1}\{x_m = k\}$ and define the empirical success probability $\phi^G_{m,k}(t)$ for every $G \in [M]$ at each time slot t as follows:

$$\phi^G_{m,k}(t) = \frac{\psi^G_{m,k}(t)}{N_{m,k}(t)} \tag{4}$$

where $N_{m,k}(t) = \sum_{s=1}^{t} \mathbb{1}\{x_m(s) = k\}$ is the number of times server k has been selected by tenant m up to round t. Variable $\psi^G_{m,k}(t)$ counts the number of times tenant

Algorithm 1 *O2C Algorithm* (for tenant $m = 1, ..., M$)

Input: K: Number of servers, M: Number of tenants, T: Time horizon, $\vec{L}$: Latency requirements for tenants, $\vec{R}$: Reliability requirements for tenants, $\mathbf{C}$: Cost matrix

$\mathbf{S}, \mathbf{N} \leftarrow$ *Zeros(M,K)*, $\boldsymbol{\psi}, \boldsymbol{\phi} \leftarrow$ *Zeros(M,K,M) # initialization*
for $t = 1, ..., T$ **do**
  if $t \leq K$ **then**
    $x_m(t) \leftarrow t$ *# Try all servers*
  else
    $x_m(t) \leftarrow$ *Choose_server*$(\boldsymbol{\phi_m}, K, M, \vec{R}, \mathbf{C})$
  end if
  Observe $d_{m,x_m(t)}(t)$, $G_{x_m(t)}(t)$
  for $g = 1, ..., M$ **do**
    if $\frac{d_{m,x_m(t)}(t)}{G_{x_m(t)}(t)} \cdot g \leq L^m$ **then**
      $\psi^g_{m,x_m(t)} \leftarrow \psi^g_{m,x_m(t)} + 1$
    end if
  end for
  Cumulative cost: $S_{m,x_m(t)} \leftarrow S_{m,x_m(t)} + C_{m,x_m(t)}$
  Number of trials: $N_{m,x_m(t)} \leftarrow N_{m,x_m(t)} + 1$
  # Empirical success probability:
  $\phi^g_{m,x_m(t)} \leftarrow \frac{\psi^g_{m,x_m(t)}}{N_{m,x_m(t)}}$
end for

m would have satisfied its latency requirements, if there were G tenants on resource k. We can compute $\psi^G_{m,k}(t)$ by looking at the past experienced delays $d_{m,k}(s), s \leq t$, computing $D_{m,k}(s) = d_{m,k}(s)/G_k(s)$ and setting $\psi^G_{m,k}(t) = \sum_{s \in \mathcal{N}_{m,k}(t)} \mathbb{1}\{D_{m,k}(s) \cdot G \leq L^m\}$.

At each time slot $t-1$, once the central controller selects server k for tenant m, it observes a corresponding constant cost $C_{m,k}$ and random delay $d_{m,k}$. At time t, for all tenants m and all possible G, we calculate $\phi^G_{m,k}(t)$ then we update Kullback-Leibler divergence upper confidence bound $\hat{\phi}^G_{m,k}(t)$ of $\phi^G_{m,k}(t)$ by the following:

$$\hat{\phi}^G_{m,k}(t) = \max\{q \leq 1 : N_{m,k}(t) \cdot KL\left(\phi^G_{m,k}(t), q\right) \leq f(t)\}$$

then we compare $\hat{\phi}^G_{m,k}(t)$ with the reliability requirement R^m. This comparison helps determine, for each tenant m and server k, *acceptable number of tenants* $\hat{G}_{m,k}$, which we define as the maximum number of tenants that can co-exist with tenant m in server k based on our previous estimations, without violating constraint (2), i.e.,

$$\hat{G}_{m,k}(t) = \max\{G \in \mathbb{N} | \hat{\phi}^G_{m,k}(t) \geq R^m\} \tag{5}$$

To choose server $x_m(t)$ to associate to tenant m in time slot t, we solve the following Integer Linear Program:

$$\begin{aligned} &\min_{x_{m,k} \in \{0,1\}} \sum_{m=1}^{M} \sum_{k=1}^{K} C_{m,k} x_{m,k} \text{ s.t. } \sum_{k=1}^{K} x_{m,k} = 1, \forall m \\ &\text{and} \quad \sum_{m'=1}^{M} x_{m',k} - \hat{G}_{m,k}(t) \leq M \cdot (1 - x_{m,k}), \forall k \end{aligned} \tag{6}$$

where decision variable $x_{m,k} = 1$ if k is associated to m and 0 otherwise.

The pseudo-code of O2C is presented in Algorithm 1. The algorithm takes as inputs: K (the number of servers), M (the number of tenants), L^m (the latency requirements for tenant m), R^m (the reliability requirements for tenant m) and $\mathbf{C}$= $(C_{m,k})_{m \in [M], k \in [K]}$ (the cost matrix). In fact, the cost matrix need not be known since it suffices to explore all servers in the first iterations to learn their values, since they are and deterministic. In Algorithm 1, at each time t, the central controller chooses servers for tenants, simultaneously observes the delays, and updates the empirical success probability $\phi^G_{m,k}$, via (4), which will be used to support the choices of the servers in the next timeslot.

The choice of the server associated to each tenant at each timeslot is detailed in Algorithm 2, instead of using empirical success probability $\phi^G_{m,k}$ directly, the controller computes *upper confidence bound* for $\phi^G_{m,k}$, constructed such that, with high probability, $\hat{\phi}^G_{m,k} \geq \phi^G_{m,k}$. The algorithm design forces the controller to be conservative in order to ensure that the chosen allocation respects the constraints with high probability and makes decisions with low regret when sufficient exploration has occurred. The regret is defined as the difference between the total cost obtained by all tenants and the cost obtained by an oracle that plays the optimal decision, denoted as $x^\star = \{x^\star_1, ..., x^\star_M\} \in \mathcal{X}$, minimizing the total cost while respecting all the chance constraints.

$$R_T = \sum_{t=1}^{T} \sum_{m=1}^{M} \left(C_{m,x_m(t)} - C_{m,x^\star_m}\right)$$

The controller computes the estimation $\hat{G}_{m,k}$ of the maximum number of concurrent users that m can coexist with server k and solves the Integer Linear Programming (ILP) problem (6). Finally, the controller executes the action based on the output of the ILP problem.

Algorithm 2 *Choose_server*(for tenant $m = 1, ..., M$)

Input: $\boldsymbol{\phi_m}$: Empirical success probability for tenant m, K: Number of servers, M: Number of tenants, $\vec{R}$: Reliability requirements for tenants, $\mathbf{C}$: Cost matrix, $\mathbf{N}$: Number of trials, t: Current time slot

Output: $\vec{x}$: Allocation for all tenants

$\hat{\mathbf{G}} = Zero(M, K)$
for $g = 1, ..., M$ **do**
  for $k = 1, ..., K$ **do**
    $\hat{\phi}^g_{m,k} = \max\{q \in [0,1] : N_{m,k} \cdot KL\ (\phi^g_{m,k}, q) \leq f(t)\}$
    if $\hat{\phi}^g_{m,k} \geq R^m$ **then** $\hat{G}_{m,k} \leftarrow g$ **end if**
  end for
end for
Solve the ILP problem (6) and output $\vec{x}$
RETURN $\vec{x}$

V. Theoretical analysis

In this section, we provide theoretical performance guarantees for our algorithm O2C, presented as Algorithm 1. We say that an algorithm is admissible if it satisfies the chance constraint of optimization problem (2)

$$\frac{1}{T}\sum_{t=1}^{T} F_{\mathcal{D}_{m,x_m(t)}}\left(\frac{L^m}{G_{x_m(t)}}\right) \geq R^m, \forall m$$

Firstly, we show in Theorem. V.1 that O2C is admissible with high probability. We then derive a regret upper bound for O2C presented in Theorem. V.2 to ensure decisions chosen by the controller converge to the optimal decision. We use the following notation. We define an acceptable number of tenants for each tenant m under the optimal decision $x^\star$:

$$G^\star_{m,x^\star_m} = \max\{G \in \mathbb{N} | F_{\mathcal{D}_{m,x^\star_m}}(L^m/G) \geq R^m\}$$

We denote by $\mu^\star_{m,L^m} = F_{\mathcal{D}_{m,x^\star_m}}(L^m/G^\star_{m,x^\star_m})$ the cumulative probability under optimal allocation $x^\star_m$ for tenant m with latency requirement L^m. For any allocation $\vec{x} = \{x_1, ..., x_M\}$ we write $\mu^{G_{m,x_m}}_{m,L^m} = F_{\mathcal{D}_{m,x_m}}\left(\frac{L^m}{G_{m,x_m}}\right)$. Our first result is Theorem V.1, showing that O2C is admissible with high probability when $T \to \infty$.

Theorem V.1. *There exists C independent of T such that*

$$\mathbb{P}(\textit{O2C is admissible}) \geq 1 - \frac{C\log(T)}{KL(\mu_{m,k}, R^m)T} \tag{7}$$

By corollary $\lim_{T\to\infty}\mathbb{P}(\textit{O2C is admissible}) = 1$.

Proof. Define the error event at round t:

$$\mathfrak{O}(t) = \bigcup_{(m,k,G)\in\mathcal{G}} \left\{\hat{\phi}^G_{m,k}(t) \geq R^m\right\}$$

$$\textit{where } \mathcal{G} = \{(m,k,G) : F_{\mathcal{D}_{m,k}}(\frac{L^m}{G_k(t)}) < R^m\}$$

This event means that an infeasible allocation was falsely considered as acceptable by the algorithm. If $\mathfrak{O}(t)$ does not occur, the ILP only uses truly feasible capacities, hence the round-t allocation is admissible. Therefore,

$$\mathbb{P}(\text{O2C is admissible}) \geq 1 - \frac{\sum_{t=1}^{T}\mathbb{P}(\mathfrak{O}(t))}{T}$$

Define $\ddot{G}_{m,x_m} = \arg\min_{G_{m,x_m}}(R^m - \mu^{G_{m,x_m}}_{m,L^m} | R^m \geq \mu^{G_{m,x_m}}_{m,L^m})$ the minimum of non-acceptable number of tenants competing for resources among all possible G_{m,x_m} for server x_m. Since each tenant offloads task to server simultaneously, $\mathfrak{O}(t) = \left\{\hat{\phi}^{\ddot{G}_{m,x_m}}_{m,k}(t) \geq R^m\right\}$ occurs until tenant m distinguishes the acceptable number of tenants $\ddot{G}_{m,x_m}$ competing for resources among all possible G_{m,x_m}. Define $\hat{\mu}_{m,k}(t) = \phi^{\ddot{G}_{m,x_m}}_{m,k}(t)$, $\mu_{m,k} = \mu^{\ddot{G}_{m,x_m}}_{m,L^m}$ and let:

$$K_{m,T} = \left\lfloor \frac{1+\epsilon}{KL(\mu_{m,k}, R^m)} f(T) \right\rfloor$$

Then, refer to Lemma. 7 in [3], we have

$$\begin{aligned}
&\sum_{t=1}^{T}\mathbb{P}(\mathfrak{O}(t)) = \mathbb{E}\left[\sum_{s=1}^{T}\mathbb{1}\left\{s \cdot KL(\hat{\mu}_{m,k}(s), R^m) < f(T)\right\}\right] \\
&= \sum_{s=1}^{T}\mathbb{P}\left(s \cdot KL(\hat{\mu}_{m,k}(s), R^m) < f(T)\right) \\
&\leq K_{m,T} + \sum_{s=K_T+1}^{\infty}\mathbb{P}\left(s \cdot KL(\hat{\mu}_{m,k}(s), R^m) < f(T)\right) \\
&= K_{m,T} + \sum_{s=K_T+1}^{\infty}\mathbb{P}\left(KL(\hat{\mu}_{m,k}(s), R^m) < \frac{KL(\mu_{m,k}, R^m)}{1+\epsilon}\right) \\
&\leq \frac{1+\epsilon}{KL(\mu_{m,k}, R^m)} f(T) + \frac{C_2(\epsilon)}{T^{\beta(\epsilon)}}
\end{aligned} \tag{8}$$

where $C_2(\epsilon), \beta(\epsilon)$ denote positive functions of ϵ. Therefore, we obtain the following with C that does not depend on T

$$\mathbb{P}(\text{O2C is admissible}) \geq 1 - \frac{C\log(T)}{KL(\mu_{m,k}, R^m)T}$$

□

We now provide an upper bound on the regret of O2C shown in Theorem V.2.

Theorem V.2. *Define $\mu^{\ddot{G}_{m^\star,k}}_{m^\star,L^{m^\star}} = \min\{\mu^{\ddot{G}_{1,k}}_{1,L^1}, ..., \mu^{\ddot{G}_{M,k}}_{M,L^M}\}$ the closest cumulative probability to R^m for server k among all tenants. The regret of the O2C algorithm satisfies the following upper bound:*

$$\limsup_{T\to\infty}\frac{\mathbb{E}[R_T]}{\log(T)} \leq \sum_{m=1}^{M}\sum_{k\neq x^\star_m}\frac{(C_{m,k} - C_{m,x^\star_m})}{KL(\mu^{\ddot{G}_{m^\star,k}}_{m^\star,L^{m^\star}}, R^{m^\star})}$$

Proof. To find the optimal decision $x^\star$ we first need to correctly estimate the acceptable number $G_{m,k}$ of tenants competing for resources (5). Let us consider, for each tenant m, an optimal server $x^\star_m$ and a sub-optimal server $x_m \neq x^\star_m$. Note that the optimal allocation is obtained if and only if all tenant select their $x^\star_m$. For any server x_m and any tenant m under any possible $G_{m,x_m} \in \{1, ..., M\}$. Let us recall $\phi^G_{m,k}(t), \psi^G_{m,k}(t), N_{m,k}(t), \hat{\phi}^G_{m,k}(t)$ defined in Sec. IV, we also

define $N_T(x_m)$ as the number of times that sub-optimal server x_m has been chosen within time horizon T by tenant m and let $u_{x^\star_m}(t) = \hat{\phi}^{G^\star_{m,x^\star_m}}_{m,x^\star_m}$.The expectation of $N_T(x_m)$ is upper-bounded using the following decomposition:

$$\begin{aligned}\mathbb{E}[N_T(x_m)] &= \mathbb{E}\left[\sum_{t=1}^{T} \mathbb{1}\{x_m(t) = x_m\}\right] \\ &\leq \mathbb{E}\left[\sum_{t=1}^{T} \mathbb{1}\{\mu^\star_{m,L^m} > u_{x^\star_m}(t)\}\right] \\ &+ \mathbb{E}\left[\sum_{t=1}^{T} \mathbb{1}\{x_m(t) = x_m, \mu^\star_{m,L^m} < u_{x^\star_m}(t)\}\right]\end{aligned}$$

The first part of the above inequality signifies that the upper confidence bound $u_{x^\star_m}(t)$ is smaller than the true value of optimal server $x^\star_m$, This is due to the fact that $\phi^{G^\star_{m,x^\star_m}}_{m,k}(t)$ is estimated not accurately at time t. The second part indicates that the upper bound of estimation for sub-optimal servers is greater than the upper bound of estimation for the optimal server, and even the upper bound of estimation for the optimal server is accurate. The reason for this is that the expression $\hat{\phi}^{G_{m,x_m}}_{m,k}(t)$ is inaccurate if the exploration of x_m is insufficient. From Theorem 10 in [3], we have the following inequality:

$$\mathbb{P}(\mu^\star_{m,L^m} > u_{x^\star_m}(t)) \leq \frac{e\lceil \log(t)^2 + 3\log(t)\log(\log(t))\rceil}{t\log(t)^3}$$

Thus, $\mathbb{P}(\mu^\star_{m,L^m} > u_{x^\star_m}(t))$ is very small and

$$\begin{aligned}&\mathbb{E}\left[\sum_{t=1}^{T} \mathbb{1}\{\mu^\star_{m,L^m} > u_{x^\star_m}(t)\}\right] \\ &= \sum_{t=1}^{T} \mathbb{P}\left(\mu^\star_{m,L^m} > u_{x^\star_m}(t)\right) \\ &= \sum_{t=1}^{T}\sum_{m} \mathbb{P}(\mu_{x^*_m} > u_{x^*_m}(t)) \\ &\leq M\sum_{t=1}^{T} \frac{e\lceil \log(t)^2 + 3\log(t)\log(\log(t))\rceil}{t\log(t)^3} \\ &\leq MC_1\log(\log(T))\end{aligned}$$

where C_1 is a positive universal constant. Combine with the inequality (8) from proof of Theorem. V.1. Hence, we have for any positive T:

$$\mathbb{E}[N_T(x_m)] \leq \frac{(1+\epsilon)\log(T)}{KL(\mu^{\ddot{G}_{m,x_m}}_{m,L^m}, R^m)} + MC_1\log(\log(T)) + \frac{C_2(\epsilon)}{T^{\beta(\epsilon)}}$$

Therefore,

$$\limsup_{T\to\infty} \frac{\mathbb{E}[N_T(x_m)]}{\log(T)} \leq \frac{1}{KL(\mu^{\ddot{G}_{m,x_m}}_{m,L^m}, R^m)}$$

When it comes to the upper bound for total regret up to T, we have the following regret for tenant m:

$$\mathbb{E}[R^m_T] \leq \sum_{x_m \neq x^\star_m} \mathbb{E}[N_T(x_m)] \cdot (C_{m,k} - C_{m,x^\star_m})$$

In our setting, note that even if tenant m has accurately estimated a sub-optimal server k, they might still choose this server because the actions of other tenants can influence their decision-making, potentially leading to a sub-optimal choice for tenant m. In the worst-case scenario, where only server k is accurately estimated by all tenants, it is certain that this sub-optimal server k will not be selected. Thus, the upper bound of total regret becomes the following:

$$\limsup_{T\to\infty} \frac{\mathbb{E}[R_T]}{\log(T)} \leq \sum_{m=1}^{M}\sum_{k\neq x^\star_m} \frac{(C_{m,k} - C_{m,x^\star_m})}{KL(\mu^{\tilde{G}_{m^\star,k}}_{m^\star,L^{m^\star}}, R^{m^\star})}$$

□

VI. Numerical results

In this section, we conduct numerical experiments to evaluate the performance of our O2C algorithm under task offloading scenario. We show the cumulative regret and reliability under different settings in the context of offloading.

We consider the case of tasks offloading where tenants can choose to send their tasks to either the cloud or the edge. In our setup, we consider $K = 2$ servers and $M = 5$ tenants. Their latency requirements are all equal to 0.2 and the reliability vector is $[0.98, 0.99, 0.95, 0.96, 0.94]$. Moreover, in task offloading, the cost of tenants for using cloud resources is usually lower compared to edge computing. However, the delay experienced by tenants is typically greater than that in edge environments. Thus, we set their cost and the mean of delay following an exponential distribution as: $\begin{bmatrix}0.3 & 0.5 & 0.6 & 0.1 & 0.2\\ 0.5 & 0.7 & 0.9 & 0.5 & 0.3\end{bmatrix}$ and $\begin{bmatrix}0.015 & 0.019 & 0.021 & 0.03 & 0.015\\ 0.01 & 0.015 & 0.019 & 0.01 & 0.01\end{bmatrix}$, respectively.

Note that, regardless of the underlying delay distribution, our O2C algorithm can identify the optimal selection while satisfying all chance constraints. Here, we evaluate our algorithm using a delay distribution modeled by an exponential distribution. We average the regret and reliability estimates over 50 independent runs in a "soft" setting and 20 independent runs in a "strict" setting, with 95% confidence intervals presented. For the sake of convenience, we take the absolute value of cumulative regret in the following figures. This is because the optimal solution that satisfies all chance constraints is generally not the one with the lowest cost. Our O2C algorithm begins with the least costly but unfeasible allocation, ensuring that the cumulative regret is always negative. However, the key focus is to demonstrate that the O2C algorithm consistently explores within feasible sets and rapidly identifies the optimal solution.

Table. I shows the cumulative regret on the above setting. Obviously, under soft setting, the O2C algorithm converges very quickly, and the cumulative regret is extremely low. This is because $\mu^{\tilde{G}_{m^\star,k}}_{m^\star,L^{m^\star}}$ (in Theorem. V.2) is not close to $R^{m^\star}$, meaning that it is not challenging to distinguish between the quality of servers. Moreover, we modified the reliability for tenants to force $\mu^{\tilde{G}_{m^\star,k}}_{m^\star,L^{m^\star}}$ closer to $R^{m^\star}$, which we call "strict"

setting. In this setting, the reliability requirement is set as $[0.99, 0.97, 0.95, 0.9, 0.98]$. As shown in Tab. I, the cumulative regret is very high, almost 100 times higher than under the "soft" setting. In addition, it converges much more slowly than in the soft setting since it is quite difficult to distinguish between the quality of some servers. Nevertheless, Tab. I under strict setting confirms our theoretical results and the O2C algorithm can still converge within acceptable rounds.

Time slot (t)	**Regret (mean±CI)**
2×10^2	51.554 ± 2.235
4×10^3	99.624 ± 6.835
8×10^3	99.624 ± 6.835
1.2×10^4	99.624 ± 6.835
1.6×10^4	99.624 ± 6.835
2×10^4	99.624 ± 6.835

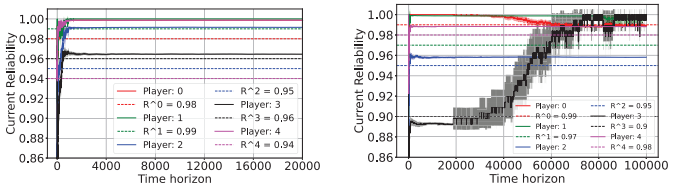
(a) Soft Setting

Time slot (t)	**Regret (mean±CI)**
10^3	$2.59 \times 10^2 \pm 11.398$
2×10^4	$4.06 \times 10^3 \pm 11.398$
4×10^4	$7.39 \times 10^3 \pm 5.04 \times 10^2$
6×10^4	$9.23 \times 10^4 \pm 1.21 \times 10^4$
8×10^4	$1.00 \times 10^4 \pm 1.71 \times 10^3$
10^5	$1.01 \times 10^4 \pm 1.79 \times 10^3$

(b) Strict Setting

TABLE I: Cumulative regret under soft and strict settings.

Fig. 1a confirms that every tenant starts exploring at the beginning and keeps exploiting edge or cloud just after hundreds of rounds. Additionally, the selections of tenants are all above their reliability requirements except for the warm-up rounds (tenants start by exploring cloud or edge to collect experience, in these rounds, they may violate their requirements). Fig. 1b



(a) Soft setting (b) Strict setting

Fig. 1: Reliability performance under (a) soft and (b) strict setting.

clearly shows that tenant (player) 3 puts lots of effort into exploring the cloud because cloud computing costs less and its reliability requirement is not strict. Other tenants' selections always lead to their reliability satisfaction. In general, optimal decisions are obtained after servers are well estimated by tenant 3. Thus, the weakness of O2C algorithm is that it may not work if $\mu^{\tilde{G}_{m^\star,k}}_{m^\star,L^{m^\star}}$ is extremely close to $R^{m^\star}$, which is a general problem in multi-bandit best-arm identification.

VII. Conclusion

In this paper, we address the chance-constrained multi-player multi-armed bandit problem under stochastic environments and propose the O2C algorithm. We derive a regret upper bound and validating its performance through extensive simulations on realistic task-offloading scenarios. The results show that O2C satisfies the chance constraints while achieving low regret, enabling efficient and reliable task offloading among multiple tenants under stochastic delays and resource contention. Future work will explore distributed decision-making schemes beyond a centralized controller.

VIII. Acknowledgment

This paper is supported by the CHIST-ERA Wireless AI 2022 call MLDR project (ANR-23-CHR4-0005), partially funded by AEI and NCN under projects PCI2023-145958-2 and 2023/05/Y/ST7/00004, respectively.

References

[1] H. E. Robbins, "Some aspects of the sequential design of experiments," *Bulletin of the American Mathematical Society*, 1952.
[2] P. Auer *et al.*, "Finite-time analysis of the multiarmed bandit problem," *Mach. Learn.*, 2002.
[3] A. Garivier and O. Cappé, "The kl-ucb algorithm for bounded stochastic bandits and beyond," in *24th COLT*. JMLR Workshop and Conference Proceedings, 2011.
[4] L. Li *et al.*, "A contextual-bandit approach to personalized news article recommendation," in *19th International Conference on World Wide Web*, 2010.
[5] P. Auer *et al.*, "The nonstochastic multiarmed bandit problem," *SIAM Journal on Computing*, 2002.
[6] S. Agrawal and N. Devanur, "Linear contextual bandits with knapsacks," *NeurIPS*, 2016.
[7] A. Badanidiyuru *et al.*, "Bandits with knapsacks," *JACM*, 2018.
[8] W. Chu *et al.*, "Online optimal service caching for multi-access edge computing: A constrained multi-armed bandit optimization approach," *Computer Networks*, 2024.
[9] A. Charnes and W. W. Cooper, "Chance-constrained programming," *Management science*, 1959.
[10] S. Jain *et al.*, "Chance constrained day ahead stochastic unit commitment with multiple uncertainties," *Journal of Electrical Engineering & Technology*, 2024.
[11] H. Zhang *et al.*, "Data-driven chance-constrained regulation capacity offering for distributed energy resources," *IEEE Transactions on Smart Grid*, 2018.
[12] T. Dinh *et al.*, "Exact algorithms for the chance-constrained vehicle routing problem," *Mathematical Programming*, 2018.
[13] M. Satyanarayanan, "The emergence of edge computing," *Computer*, 2017.
[14] Y. Huang *et al.*, "Task scheduling with optimized transmission time in collaborative cloud-edge learning," in *ICCCN*, 2018.
[15] M. Chen and Y. Hao, "Task offloading for mobile edge computing in software defined ultra-dense network," *IEEE JSAC*, 2018.
[16] L. Huang *et al.*, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE TMC*, 2019.
[17] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE TVT*, 2018.
[18] M. Tang and V. W. Wong, "Deep reinforcement learning forxs task offloading in mobile edge computing systems," *IEEE TMC*, 2020.
[19] L. Kleinrock, "Time-shared systems: A theoretical treatment," *Journal of the ACM (JACM)*, vol. 14, no. 2, pp. 242–261, 1967.
[20] Amazon Web Services, "Amazon web services service level agreements," https://aws.amazon.com/legal/service-level-agreements, 2022.
[21] R. Atawia *et al.*, "Joint chance-constrained predictive resource allocation for energy-efficient video streaming," *IEEE JSAC*, 2016.